

Activités d'Intégration de Compétences —

Partie 3

Objectif général

Faire évoluer le projet vers une architecture **micro services** complète en introduisant la découverte de services (Eureka), la communication inter-services (Feign), une API Gateway, un second micro service de gestion des notes, un frontend Next.js, et une application mobile enrichie. Le tout est orchestré via un `docker-compose.yml` unifié.

Q1 — Créer une branche Git `version-3` et ajouter le Sprint 3 dans Jira

Indication : Comme pour la Partie 2, isolez votre travail dans une branche dédiée créée à partir de `version-2`. Parallèlement, ouvrez votre projet Jira Scrum et créez un **Sprint 3** dans le backlog. Décomposez les questions ci-dessous en User Stories et Tasks avant de commencer le développement — c'est une bonne pratique Agile que vous pratiquez ici concrètement.

```
git checkout version-2
git checkout -b version-3
```

Q2 — Définir le workflow GitHub

Indication : Un workflow GitHub structuré est essentiel dans tout projet collaboratif professionnel. L'objectif ici est de mettre en place des règles et des modèles qui garantissent la qualité du code avant toute fusion dans les branches principales. Configurez les éléments suivants dans votre dépôt :

Protection de branche : dans les paramètres GitHub de votre dépôt (`Settings` > `Branches`), activez la protection sur `main` et `version-3` : exigez au moins une review approuvée avant tout merge, et interdisez les pushes directs.

Template d'issue : créez le fichier `.github/ISSUE_TEMPLATE/bug_report.md` (et/ou `feature_request.md`) pour standardiser le signalement de bugs et de nouvelles fonctionnalités.

Template de Pull Request : créez `.github/pull_request_template.md` avec des sections prédéfinies : description du changement, lien vers le ticket Jira, checklist de review (tests passants, documentation mise à jour, etc.).

Convention de review : rédigez dans votre `README.md` une section décrivant les règles de review adoptées par l'équipe (ex. : toute PR doit être relue dans les 48h, les commentaires bloquants doivent être résolus avant le merge, etc.).

Q3 — Ajouter le micro service `grading-service` (Gestion des Notes)

Indication : Créez un nouveau projet Spring Boot indépendant nommé `grading-service`. Ce micro service gère l'entité `Note` et communique avec le micro service étudiant pour valider l'existence d'un étudiant avant d'enregistrer une note. Respectez la même architecture en couches que dans la Partie 2 (controller, service, repository, entity, dto, mapper, config).

L'entité `Note` comporte les champs suivants :

Champ	Type	Description
<code>id</code>	Long	Identifiant auto-généré
<code>studentId</code>	Long	Référence vers l'étudiant (pas de FK JPA directe)
<code>matiere</code>	String	Nom de la matière
<code>valeur</code>	Double	Valeur de la note (0–20)

Implémentez les opérations CRUD complètes pour `Note`, avec gestion des erreurs HTTP standard et documentation Swagger, comme établi en Partie 2.

Q4 — Ajouter Feign, Eureka et API Gateway

Indication : Cette question introduit les trois piliers de la communication dans une architecture Spring Cloud. Voici comment les articuler :

Eureka est le serveur de registre : chaque micro service s'y enregistre au démarrage et l'interroge pour trouver les autres. Créez un projet Spring Boot dédié (`eureka-server`) avec `@EnableEurekaServer`. Chaque micro service (étudiant, notes) doit inclure `spring-cloud-starter-netflix-eureka-client` et se déclarer avec un `spring.application.name`.

Feign est le client HTTP déclaratif : dans `grading-service`, créez une interface annotée `@FeignClient(name = "etudiant-service")` avec une méthode pointant vers `GET /api/etudiants/{id}`. Feign résoudra automatiquement l'adresse via Eureka. Utilisez ce client dans le service pour vérifier qu'un étudiant existe avant d'enregistrer une note.

API Gateway (Spring Cloud Gateway) est le point d'entrée unique : créez un projet `api-gateway` qui route `/api/etudiants/**` vers `etudiant-service` et `/api/notes/**` vers `grading-service`. Les clients externes (mobile, frontend) n'appellent plus que l'API Gateway.

```
# Exemple de route dans api-gateway application.yml
spring:
  cloud:
    gateway:
      routes:
        - id: etudiant-service
          uri: lb://etudiant-service
          predicates:
            - Path=/api/etudiants/**
        - id: grading-service
          uri: lb://grading-service
          predicates:
            - Path=/api/notes/**
```

Q5 — Mettre à jour l'application mobile pour afficher les étudiants par département

Indication : L'application mobile (Flutter ou React Native) développée en Partie 1 affichait une liste plate d'étudiants. Mettez-la à jour pour d'abord récupérer la liste des départements via `GET /api/departements`, puis permettre à l'utilisateur de sélectionner un département et d'afficher uniquement les étudiants de ce département. Toutes les requêtes doivent désormais passer par l'**API Gateway** (plus directement par le micro service).

En Flutter, utilisez un `DropDownButton` pour la sélection du département et un `FutureBuilder` pour charger les étudiants de manière asynchrone. En React Native, utilisez un `Picker` (ou `FlatList` de boutons) et `useEffect` pour déclencher le fetch à chaque changement de département sélectionné.

Q6 — Créer un frontend Next.js pour la gestion des étudiants et des départements

Indication : Créez un projet Next.js dans un dossier `frontend/` à la racine du dépôt. Ce frontend doit permettre de consulter, créer, modifier et supprimer des étudiants et des départements via l'API Gateway. Organisez le projet en pages distinctes : `/etudiants` pour la liste et le formulaire d'ajout/édition, `/departements` pour la gestion des départements.

Utilisez `fetch` ou `axios` avec les Server Components de Next.js (App Router) pour les lectures, et des Client Components (`"use client"`) pour les formulaires interactifs. Stylez l'interface avec Tailwind CSS pour un rendu propre et responsive. Toutes les requêtes pointent vers l'API Gateway.

Structure de pages recommandée :

```

frontend/
├── app/
│   ├── etudiants/
│   │   ├── page.tsx           // Liste des étudiants
│   │   └── [id]/page.tsx      // Détail / édition
│   ├── departements/
│   │   └── page.tsx           // Liste et gestion des départements
│   └── layout.tsx
├── components/
│   ├── EtudiantCard.tsx
│   └── DepartementForm.tsx
└── package.json

```

Q7 — Mettre à jour le `docker-compose.yml` pour inclure tous les micro services

Indication : Le `docker-compose.yml` à la racine du projet doit désormais orchestrer l'ensemble de l'architecture. Chaque service doit être connecté au même réseau Docker et déclarer ses dépendances avec `depends_on`. Veillez à ce que `eureka-server` démarre en premier, puis les micro services métier, et enfin l'API Gateway et le frontend.

Services à inclure dans le `docker-compose.yml` :

Service	Port exposé	Dépend de
postgres	5432	—
redis	6379	—
eureka-server	8761	—
etudiant-service	8081	postgres, redis, eureka-server
grading-service	8082	postgres, eureka-server
api-gateway	8080	eureka-server
frontend	3000	api-gateway

```
# Extrait docker-compose.yml
services:
  eureka-server:
    build: ./eureka-server
    ports:
      - "8761:8761"

  etudiant-service:
    build: ./api-spring-boot
    ports:
      - "8081:8081"
    depends_on:
      - postgres
      - redis
      - eureka-server
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/etudiants
      EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://eureka-server:8761/eureka

# ... autres services
```

Structure du dépôt GitHub attendue

```
/projet-etudiants/
├── api-spring-boot/           # Micro service étudiant
├── grading-service/          # Micro service notes (nouveau)
├── eureka-server/            # Serveur de registre (nouveau)
├── api-gateway/              # API Gateway (nouveau)
├── frontend/                 # Application Next.js (nouveau)
├── mobile-app/               # Application Flutter / React Native (mise à jour)
├── k8s/                      # Manifests Kubernetes (Partie 2)
├── .github/
│   ├── ISSUE_TEMPLATE/
│   │   └── bug_report.md
│   └── pull_request_template.md
└── docker-compose.yml        # Mis à jour avec tous les services
```

💡 **Rappel** : chaque fonctionnalité doit faire l'objet d'une Pull Request vers `version-3`, reliée à un ticket Jira du Sprint 3. Profitez du workflow GitHub mis en place en Q2 pour pratiquer la review de code, même en solo (auto-review ou review croisée entre étudiants).